C++ Course

Materials for Students

Paweł Kisielewicz pkisiel@gmail.com

Krakow, 18 June 2026

Contents

Chapter 1

Introduction

- Author: Paweł Kisielewicz `<pkisiel@gmail.com>`

- Place and date: Krakow, 18 June 2026
- Version: `0.2.0-en`

This ebook is the first English draft based on the Polish C++ learning materials. The Polish version remains the source of truth for now. The English version starts with the course structure, learning path, and translation plan.

The full course is organized from environment setup and C++ basics to functions, arrays, strings, files, exceptions, object-oriented programming, STL, build systems, testing, and final projects.

Chapter 2

English Version Plan

2.1 Goal

The English edition should become a downloadable version of the C++ course for students who want to learn from the same material as the Polish course.

The first release should focus on readability and navigation. The code examples can stay close to the Polish repository at the beginning, while explanations, chapter introductions, exercises, and project descriptions are translated in batches.

2.2 Learning Path

1. `00-start` - development environment, compiler, editor, Git, and CMake.
2. `01-basics` - first program, input/output, types, variables, and operators.
3. `02-control-flow-loops` - conditions, `switch`, loops, `break`, `continue`.
4. `03-functions-arrays-strings` - functions, arrays, matrices, and strings.
5. `04-pointers-references-memory` - addresses, pointers, references, and safe memory habits.
6. `05-files-exceptions` - text files, validation, and exceptions.
7. `06-oop` - classes, objects, constructors, encapsulation, inheritance, and operators.
8. `07-stl-data-structures` - STL containers, algorithms, maps, stacks, queues, and lists.
9. `08-project-build-tests` - multi-file projects, build scripts, tests, and CI.
10. `09-modern-cpp` - selected modern C++ features, lambdas, move semantics, namespaces, and threads.
11. `10-projects` - final project topics, grading checklists, and project planning.

2.3 Translation Batches

2.3.1 Batch 1

- Main repository introduction - draft added.
- Ebook front matter - draft added.
- `00-start` - first English chapter added.
- `01-basics` - first English chapter added.

2.3.2 Batch 2

- `02-control-flow-loops`.
- Bridge exercise: minimum, maximum, and average.

2.3.3 Batch 3

- `03-functions-arrays-strings`.
- Bridge exercise: GADERYPOLUKI cipher.

2.3.4 Batch 4

- `04-pointers-references-memory`.
- `05-files-exceptions`.
- `06-oop`.
- Bridge exercise: figures and polymorphism.

2.3.5 Batch 5

- `07-stl-data-structures`.
- `08-project-build-tests`.
- Bridge exercise: RPN calculator.

2.3.6 Batch 6

- `09-modern-cpp`.
- `10-projects`.
- Project checklist and grading card.

2.4 Editorial Rules

- Keep the English text simple and practical.
- Do not translate code identifiers unless the example itself is rewritten.
- Prefer short paragraphs and clear section titles.
- Keep exercises close to the Polish version, but adjust wording for an English-speaking student.
- Do not include archive materials in the ebook.
- Keep the Polish and English ebook versions buildable by the same script.

2.5 YouTube Support

The first bilingual video format should be short quizzes:

- Polish question first.
- English question second.
- One C++ concept per video.
- Link to the repository and both ebook downloads in the description.

Example:

```
C++ Quiz #1
What will this program print?

int x = 5;
std::cout << x++ << " " << x;

A) 5 5
B) 5 6
C) 6 6
```

Answer: `B) 5 6`, because postfix `x++` returns the current value first and increments the variable afterwards.

2.6 First Downloadable English Draft

The first downloadable English file should be marked clearly as a draft:

- title: `C++ Course`
- version: `0.1.0-en`
- scope: plan, learning path, and translation roadmap
- output files:

- ebook/download/cpp-course-en.pdf
- ebook/download/cpp-course-en.epub

Chapter 3

00 - Start

This chapter prepares the technical environment for learning C++. It is meant for students who are starting with a compiler, code editor, terminal, Git, and simple project structure.

3.1 Learning Goals

After this chapter you should be able to:

- explain what a C++ compiler does,
- compile and run a simple `.cpp` file,
- use a terminal for basic commands,
- open and edit source files in Visual Studio Code or another editor,
- understand why Git is useful during programming exercises,
- understand the role of `CMakeLists.txt` in larger projects,
- move safely to the first C++ programming chapter.

3.2 What You Need

You need:

- a C++ compiler with C++17 support,
- a text editor or IDE,
- a terminal,
- Git,
- access to this repository.

On macOS and Linux, the compiler is often available as `c++`, `clang++`, or `g++`. On Windows, students usually use MSYS2, Visual Studio Build Tools, or an IDE configured by the teacher.

3.3 First Compiler Check

Open a terminal and run:

```
c++ --version
```

If the command prints compiler information, you can continue. If it says that the command is missing, install a compiler before moving forward.

3.4 First Program

Create a file named `hello.cpp`:

```
#include <iostream>
```

```cpp
int main() {
    std::cout << "Hello, C++!" << std::endl;
    return 0;
}
```

Compile it:

```
c++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello
```

Run it:

```
./hello
```

Expected output:

```
Hello, C++!
```

3.5 Editor

Use an editor that lets you:

- open folders,
- edit `.cpp` files,
- open an integrated terminal,
- see compiler errors clearly.

Visual Studio Code is a practical default for beginners, but the course does not depend on one specific editor.

3.6 Git Basics

Git helps you save progress and review changes. For this course, the most important commands are:

```
git status
git add file.cpp
git commit -m "Describe the change"
```

At the beginning, treat Git as a history of your learning steps. Commit small, working changes.

3.7 CMake in One Sentence

CMake is a build configuration tool. In small exercises you can compile one file manually. In larger projects, CMake or a build script describes how many source files should be compiled together.

3.8 Student Checklist

Before going to the next chapter, make sure you can:

- open a terminal,
- check the compiler version,
- compile `hello.cpp`,
- run the generated program,
- explain what source file and executable mean,
- check Git status in a repository.

3.9 Next Step

Go to chapter `01 - Basics` and write your first simple console programs.

Chapter 4

01 - C++ Basics

This chapter is the first programming block of the course. It introduces the structure of a C++ program, input and output, basic data types, variables, operators, and simple console tasks.

4.1 Learning Goals

After this chapter you should be able to:

- describe the role of an algorithm, a program, and the `main` function,
- write, compile, and run a simple C++ program,
- use `std::cout`, `std::cin`, and `std::getline`,
- choose basic data types for a simple problem,
- declare and initialize variables,
- perform simple calculations,
- format basic text output,
- solve small console exercises.

4.2 Minimal Program Structure

Most beginner programs in this course start like this:

```
#include <iostream>

int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

Important parts:

- `#include <iostream>` makes console input/output available,
- `int main()` is the starting point of the program,
- statements inside `{ }` are executed in order,
- `return 0;` means the program finished successfully.

4.3 Output

Use `std::cout` to print text:

```
#include <iostream>

int main() {
    std::cout << "C++ basics" << std::endl;
```

```
    std::cout << "Second line" << std::endl;
}
```

`std::endl` ends the current line.

4.4 Input

Use `std::cin` to read a value:

```
#include <iostream>
#include <string>

int main() {
    std::string name;

    std::cout << "Your name: ";
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;
}
```

Use `std::getline` when you want to read a full line with spaces:

```
#include <iostream>
#include <string>

int main() {
    std::string fullName;

    std::cout << "Full name: ";
    std::getline(std::cin, fullName);

    std::cout << "Hello, " << fullName << "!" << std::endl;
}
```

4.5 Basic Types

Common beginner types:

Type	Example use
`int`	whole numbers
`double`	decimal numbers
`char`	one character
`bool`	true or false
`std::string`	text

Example:

```
int age = 20;
double price = 19.99;
bool active = true;
std::string city = "Krakow";
```

4.6 Simple Calculation

```
#include <iostream>

int main() {
```

```
    double a = 0.0;
    double b = 0.0;

    std::cout << "First number: ";
    std::cin >> a;

    std::cout << "Second number: ";
    std::cin >> b;

    std::cout << "Sum: " << a + b << std::endl;
    std::cout << "Product: " << a * b << std::endl;
}
```

4.7 Formatting Output

For decimal precision, use `<iomanip>`:

```
#include <iomanip>
#include <iostream>

int main() {
    double value = 10.0 / 3.0;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << value << std::endl;
}
```

Expected output:

```
3.33
```

4.8 Common Mistakes

- forgetting `#include <iostream>`,
- missing semicolon at the end of a statement,
- using `int` when the result should contain decimals,
- mixing `std::cin >> value` and `std::getline` without understanding how input is buffered,
- ignoring compiler warnings.

4.9 Practice Tasks

1. Print your name, city, and favorite programming topic.
2. Read two integers and print their sum, difference, and product.
3. Read a rectangle width and height, then print its area.
4. Read a temperature in Celsius and print the value in Fahrenheit.
5. Read a full name with spaces and print a greeting.

4.10 Student Checklist

Before moving to control flow and loops, make sure you can:

- write a minimal C++ program,
- compile it with `-std=c++17 -Wall -Wextra -pedantic`,
- read one value from the user,
- read one full line from the user,
- choose between `int`, `double`, `bool`, and `std::string`,
- format a decimal number with two digits after the decimal point.

4.11 Next Step

The next topic is control flow: `if`, `else`, `switch`, and loops.